LAB 5. Account Management (Roles Admin)

I. User Case Dang Nhap – role customer :

cho phép người dùng tạo tài khoản và đăng nhập qua Gmail. Sử dụng oauth 2.0 aps.net core

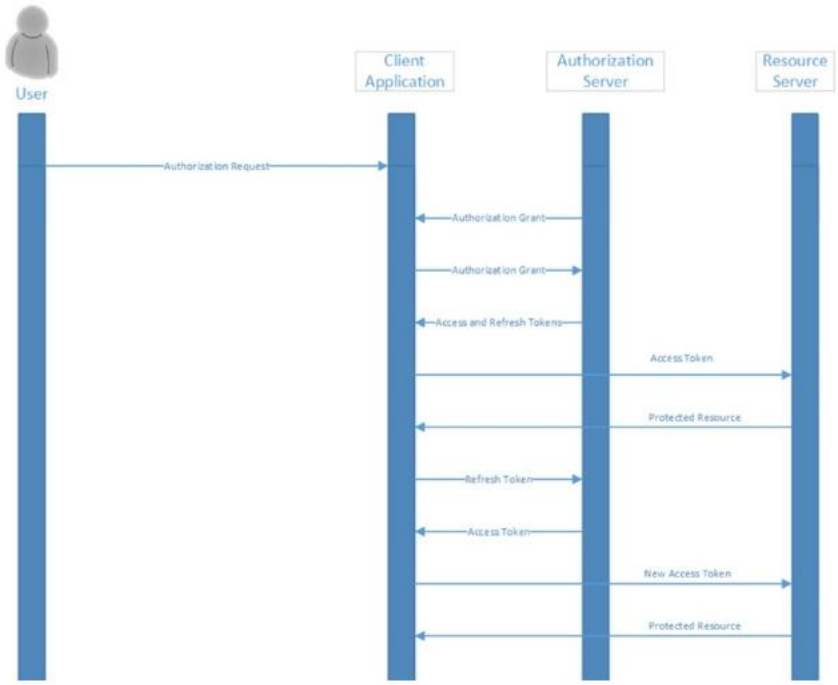
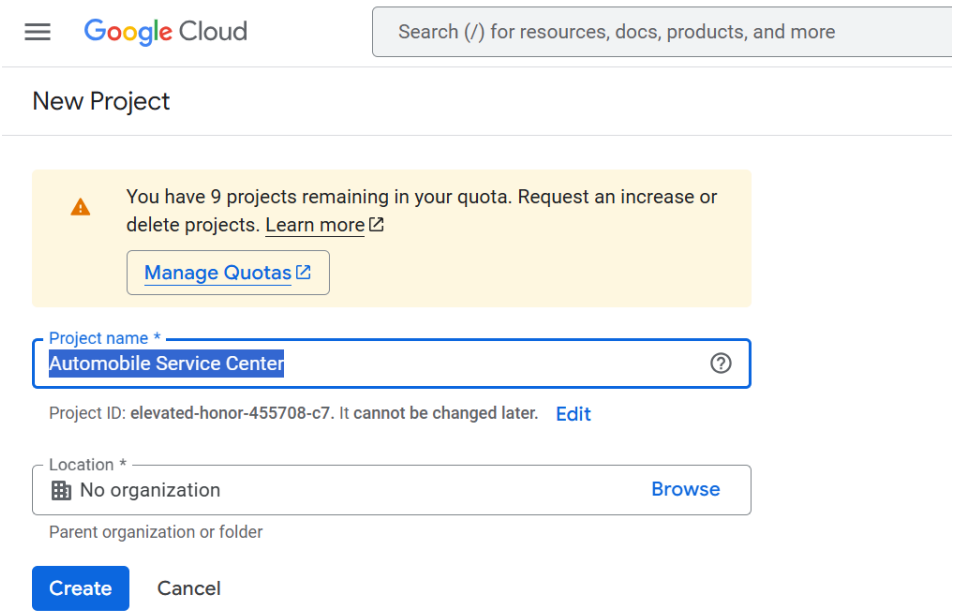
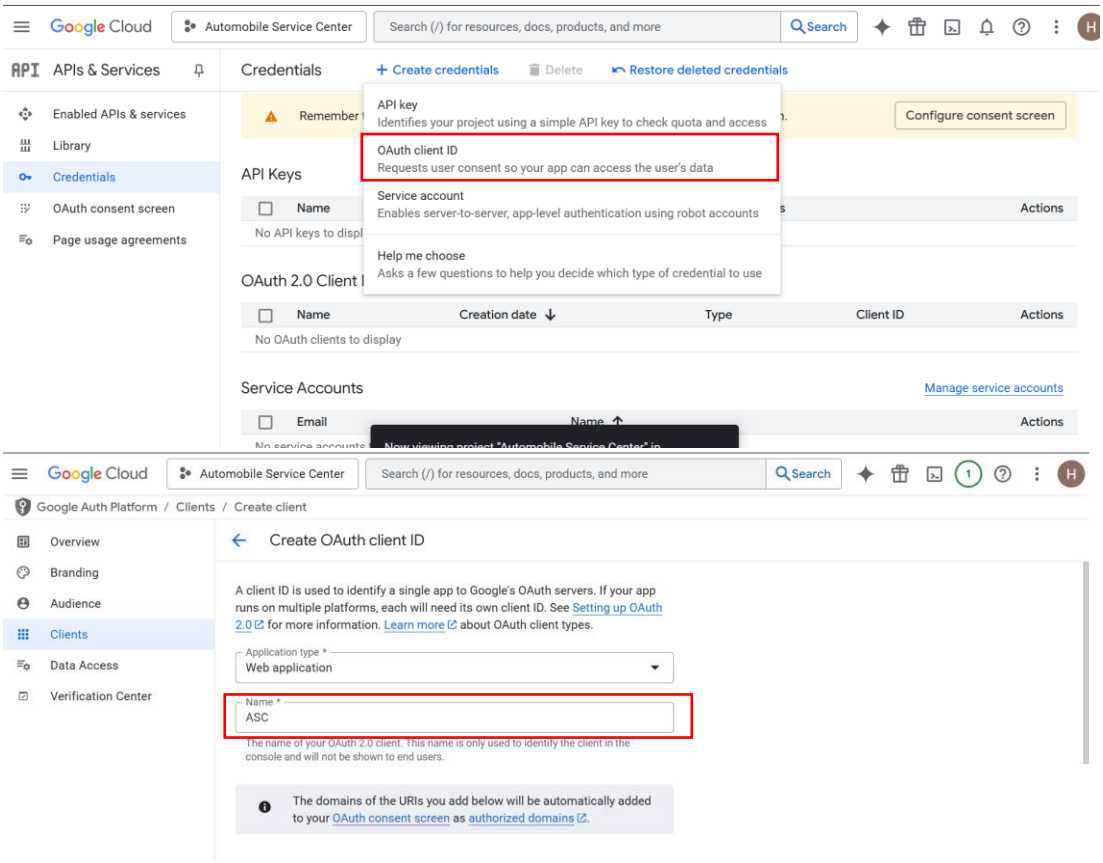


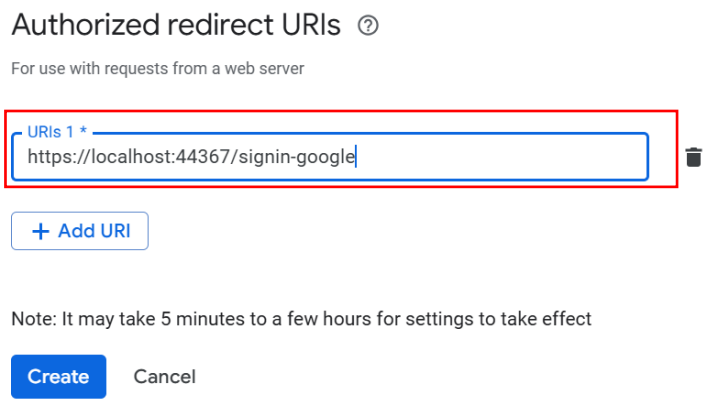
Figure 6-30. *OAuth 2.0 authorization flow*

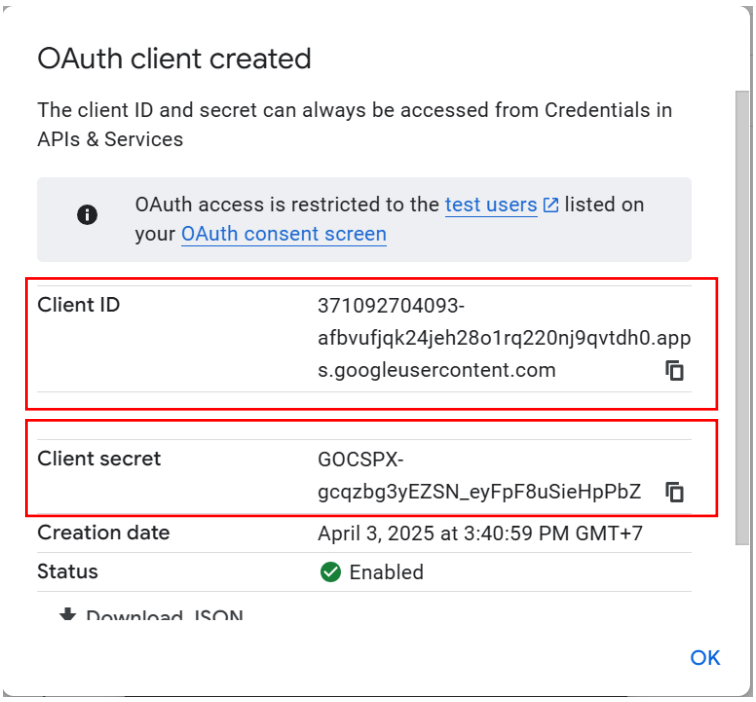
- **Link tham khảo**
- **Truy cập link và đăng nhập với gmail**
  https://console.cloud.google.com/projectselector2/apis/library?invt=AbtwFA
  Chọn create new project



- **Truy cập dự án vào tạo 1 Credentials -> Create credentials -> Oauth Client ID**



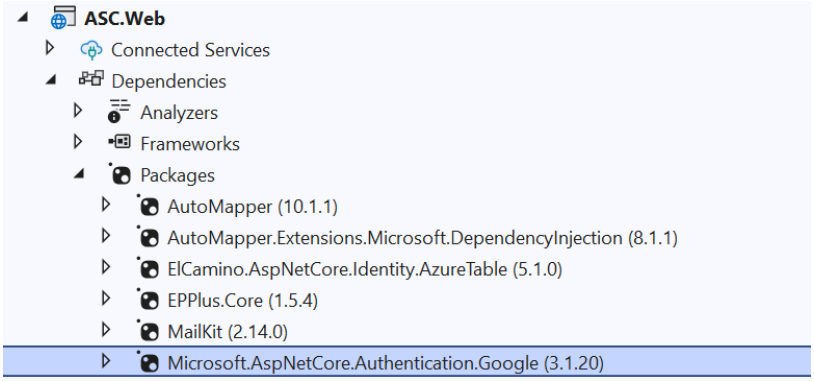




- **Cập nhập appsettings.json**

```
"Google": {
  "Identity": {
    "ClientId": "527974531025-hl9m893gihf4v1e2odk9ojuibpl89cq7.apps.googleusercontent.com",
    "ClientSecret": "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"
  }
},
```

- **Sử dụng nuget cài : Microsoft.AspNetCore.Authentication.Google**



- **Cấu hình dịch vụ:**

```
namespace ASC.Web.Services
{
    0 references | Tran Van Huu, 8 days ago | 1 author, 2 changes
    public static class DependencyInjection //
    {
        //Config services
        1 reference | Tran Van Huu, 8 days ago | 1 author, 1 change
        public static IServiceCollection AddCongfig(this IServiceCollection services, IConfiguration config) {

            // Add AddDbContext with connectionString to mirage database
            var connectionString =config.GetConnectionString("DefaultConnection") ??
                throw new InvalidOperationException("Connection string 'DefaultConnection' not found.");
            services.AddDbContext<ApplicationDbContext>(options => options.UseSqlServer(connectionString));
            //Add Options  and get data from  appsettings.json with "AppSettings"
            services.AddOptions();// IOption
            services.Configure<ApplicationSettings>(config.GetSection("AppSettings"));

            //Using a Gmail Authentication Provider for Customer Authentication
            services.AddAuthentication()
                .AddGoogle(options =>
                  {
                      IConfigurationSection googleAuthNSection = config.GetSection("Authentication:Google");
                      options.ClientId = config["Google:Identity:ClientId"];
                      options.ClientSecret = config["Google:Identity:ClientSecret"];
                  });
            return services;
        }
```

- **Cập nhật view login cho phép đăng nhập và tạo tài khoản khách hàng ./Areas/Identity/Pages/Account/Login.cshtml**

```
        <div class="col s12 m4 offset-m1">
            <h6>Login using External Providers.</h6>
            <div class="divider"></div>
            @{var loginProviders = await SignInManager.GetExternalAuthenticationSchemesAsync();
                if (loginProviders.Count() !=0)
                {
                    <form asp-area="Identity" asp-page="/Account/ExternalLogin" asp-route-returnurl="@ViewData["ReturnUrl"]" method="post" class="formhorizontal">
                        <div>
                            <p>
                                @foreach (var provider in loginProviders)
                                {
                                    <button type="submit" class="btn btndefault" name="provider" value="@provider.DisplayName" title="Log in using your@provider.DisplayName account">
                                        @provider.DisplayName
                                    </button>
                                }
                            </p>
                        </div>
                    </form>
                }
            }
        </div>
    </div>
</div>
```

- **Cập nhật view /Areas/Identity/Pages/Account/ExternalLogin.cshtml**

```
@page
@model ExternalLoginModel
@{
    ViewData["Title"] = "Register";
    Layout = "/Views/Shared/_Layout.cshtml";
}

<div class="container">
    <div class="row">
        <div class="col s12 m6 offset-m3">
            <form asp-page-handler="Confirmation" asp-route-returnUrl="@Model.ReturnUrl" method="post">
                <div asp-validation-summary="All" class="text-danger"></div>
                <p class="text-info">
                    You've successfully authenticated with <strong>@Model.ProviderDisplayName</strong>.
                    Please click the Register button to finish
                    logging in.
                </p>
                <div class="row">
                    <div class="input-field col s12">
                        <input asp-for="Input.Email"  type="email" readonly>
                        <label asp-for="Input.Email">Email</label>
                    </div>
                </div>
                <div class="row center">
                    <button class="btn waves-effect waves-light" type="submit">
                        Register
                        <i class="material-icons right">send</i>
                    </button>
                </div>
            </form>
        </div>
    </div>
</div>


@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

- **Cập nhật code xử lý /Areas/Identity/Pages/Account/ExternalLogin.cs**

```
8 references | Tran Van Huu, 14 days ago | 1 author, 1 change
public class ExternalLoginModel : PageModel
{
    private readonly SignInManager<IdentityUser> _signInManager;
    private readonly UserManager<IdentityUser> _userManager;
    private readonly ILogger<ExternalLoginModel> _logger;

    0 references | 0 changes | 0 authors, 0 changes
    public ExternalLoginModel(
        SignInManager<IdentityUser> signInManager,
        UserManager<IdentityUser> userManager,
        ILogger<ExternalLoginModel> logger)
    {
        _signInManager = signInManager;
        _userManager = userManager;
        _logger = logger;
    }
```

//Model

```
[BindProperty]
5 references | Tran Van Huu, 14 days ago | 1 author, 1 change
public InputModel Input { get; set; }
2 references | Tran Van Huu, 14 days ago | 1 author, 1 change
public string ProviderDisplayName { get; set; }
2 references | Tran Van Huu, 14 days ago | 1 author, 1 change
public string ReturnUrl { get; set; }
[TempData]
2 references | Tran Van Huu, 14 days ago | 1 author, 1 change
public string ErrorMessage { get; set; }
2 references | Tran Van Huu, 14 days ago | 1 author, 1 change
public class InputModel
{
    [Required]
    [EmailAddress]
    5 references | Tran Van Huu, 14 days ago | 1 author, 1 change
    public string Email { get; set; }
}
```

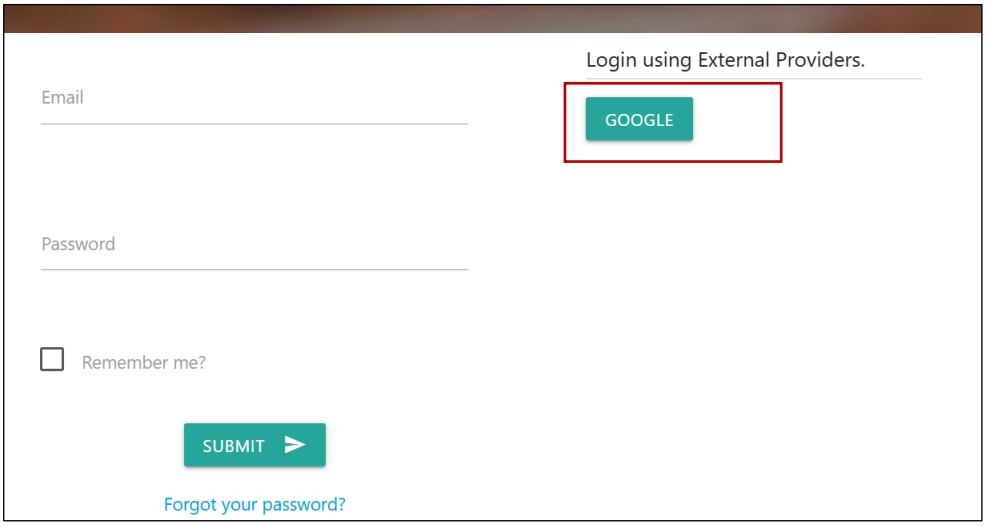
//Actions

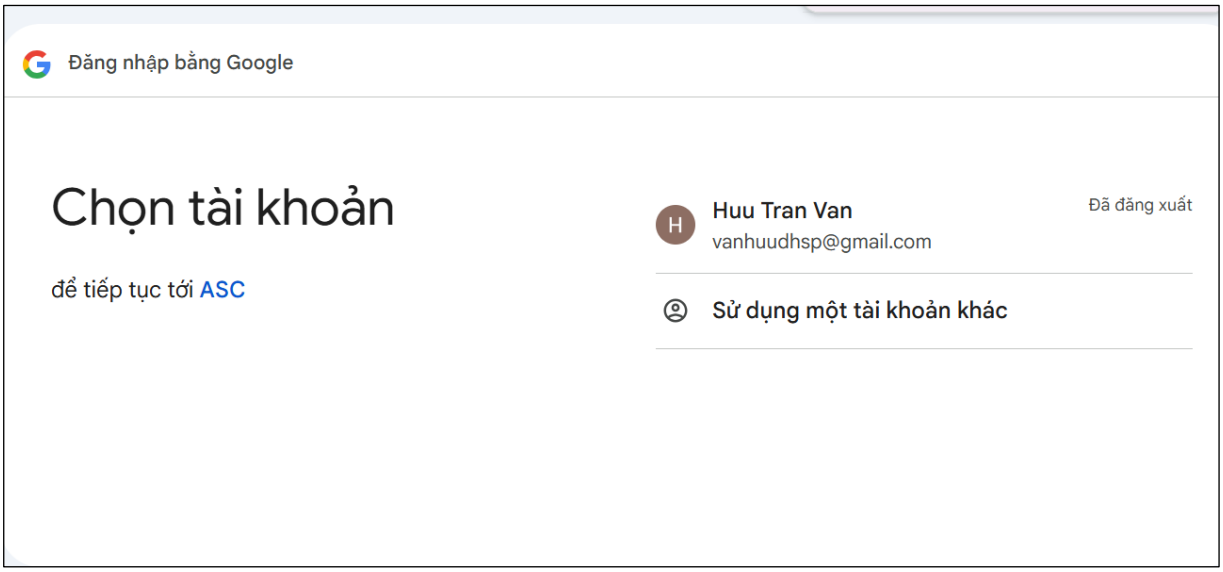
```
0 references | Tran Van Huu, 14 days ago | 1 author, 1 change
public IActionResult OnGet() => RedirectToPage("./Login");

0 references | Tran Van Huu, 14 days ago | 1 author, 1 change
public IActionResult OnPost(string provider, string returnUrl = null)
{
    // Request a redirect to the external login provider.
    var redirectUrl = Url.Page("./ExternalLogin", pageHandler: "Callback", values: new { returnUrl });
    var properties = _signInManager.ConfigureExternalAuthenticationProperties(provider, redirectUrl);
    return new ChallengeResult(provider, properties);
}
```

```
public async Task<IActionResult> OnGetCallbackAsync(string returnUrl = null, string remoteError = null)
{
    returnUrl = returnUrl ?? Url.Content("~/");
    if (remoteError != null)
    {
        ErrorMessage = $"Error from external provider: {remoteError}";
        return RedirectToPage("./Login", new { ReturnUrl = returnUrl });
    }
    var info = await _signInManager.GetExternalLoginInfoAsync();
    if (info == null)
    {
        ErrorMessage = "Error loading external login information.";
        return RedirectToPage("./Login", new { ReturnUrl = returnUrl });
    }

    // Sign in the user with this external login provider if the user already has a login.
    var result = await _signInManager.ExternalLoginSignInAsync(info.LoginProvider, info.ProviderKey, isPersistent: false, bypassTwoFactor: true);
    if (result.Succeeded)
    {
        _logger.LogInformation("{Name} logged in with {LoginProvider} provider.", info.Principal.Identity.Name, info.LoginProvider);
        return  RedirectToAction("Dashboard", "Dashboard", new { area = "ServiceRequests" });
    }
    if (result.IsLockedOut)
    {
        return RedirectToPage("./Lockout");
    }
    else
    {
        // If the user does not have an account, then ask the user to create an account.
        ReturnUrl = returnUrl;
        ProviderDisplayName = info.ProviderDisplayName;
        if (info.Principal.HasClaim(c => c.Type == ClaimTypes.Email))
        {
            Input = new InputModel
            {
                Email = info.Principal.FindFirstValue(ClaimTypes.Email)
            };
        }
        return Page();
    }
}
```

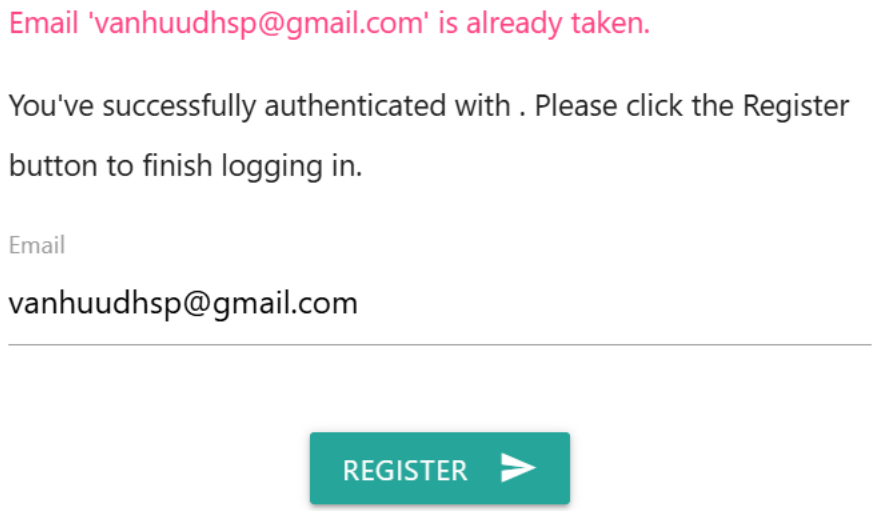
```
107         public async Task<IActionResult> OnPostConfirmationAsync(string returnUrl = null)
108         {
109
110             if (ModelState.IsValid)
111             {
112                 // Get the information about the user from the external login provider
113                 var info = await _signInManager.GetExternalLoginInfoAsync();
114                 if (info == null)
115                 {
116                     return RedirectToPage("./ExternalLoginFailure");
117                 }
118                 var user = new IdentityUser
119                 {
120                     UserName = Input.Email,
121                     Email = Input.Email,
122                     EmailConfirmed = true
123                 };
124                 var result = await _userManager.CreateAsync(user);
125                 await _userManager.AddClaimAsync(user, new System.Security.Claims.Claim("http://schemas.xmlsoap.org/ws/2005/05/identity/claims/emailaddress", user.Email));
126                 await _userManager.AddClaimAsync(user, new System.Security.Claims.Claim("IsActive", "True"));
127                 if (!result.Succeeded)
128                 {
129                     result.Errors.ToList().ForEach(p => ModelState.AddModelError("", p.Description));
130                     return Page();
131                 }
132                 // Assign user to User Role
133                 var roleResult = await _userManager.AddToRoleAsync(user, Roles.User.ToString());
134                 if (!roleResult.Succeeded)
135                 {
136                     roleResult.Errors.ToList().ForEach(p => ModelState.AddModelError("", p.Description));
137                     return Page();
138                 }
139                 if (result.Succeeded)
140                 {
141                     result = await _userManager.AddLoginAsync(user, info);
142                     if (result.Succeeded)
143                     {
144                         await _signInManager.SignInAsync(user, isPersistent: false);
145                         _logger.LogInformation(6, "User created an account using {Name} provider.", info.LoginProvider);
146                         return RedirectToAction("Dashboard", "Dashboard", new { area = "ServiceRequests" });
147                     }
148                 }
149                 ModelState.AddModelError(string.Empty, result.ToString());
150             }
151             ViewData["ReturnUrl"] = returnUrl;
152             return Page();
153         }
154     }
```

- **Chạy ứng dụng => Click google**

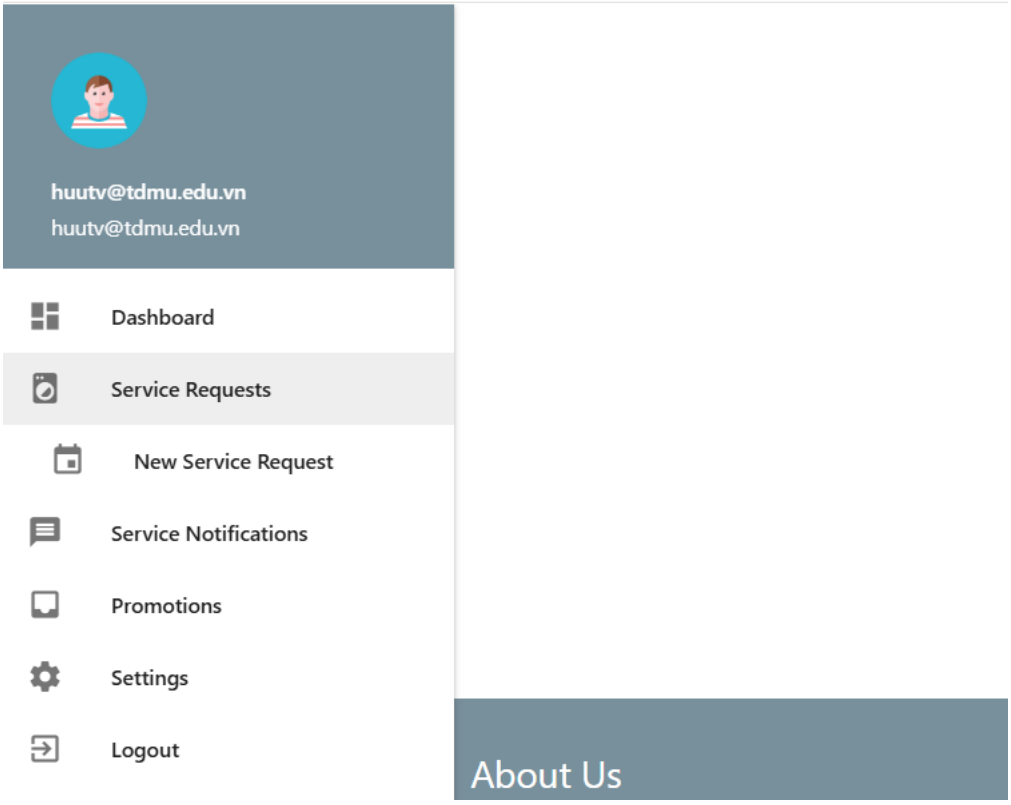




Trường hợp chưa đăng ký tài khoản qua gmail, nếu email đăng ký tài khoản Admin hoặc Engeering thông báo lỗi

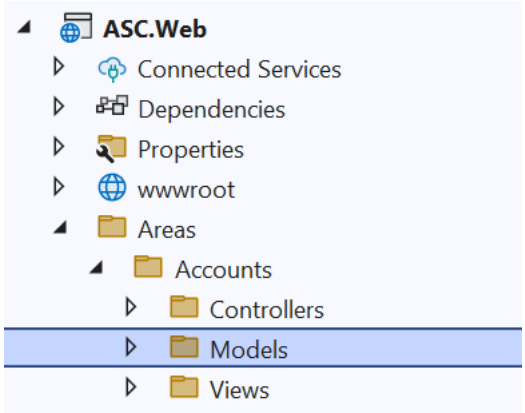


Trường hợp chưa đăng ký tài khoản qua gmail và email chưa dùng đăng ký tài khoản Admin hoặc Engeering. Sẽ tạo tài khoản cho User và chuyển sang dashboard của người dùng



II. Use Case Service Engineer Dashboard - Role Admin: quản lý tài khoản nhân viên sửa chữa. Cho phép thêm mới tài khoản nhân viên sửa chữa. Cập nhật thông tin nhân viên sửa chữa. Trường hợp nhân viên không làm thì sẽ deactive tài khoản (không xóa tài khoản). Khi cập nhật thông tin tài khoản phải gửi thống báo đến nhân viên qua email

B1. Tạo Area Accounts



Tạo layout /Areas/Accounts/Views/_ViewImports.cshtml cho phép import @addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers

```
@using ASC.Web
@using ASC.Web.Models
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

B2. Tạo các class Model

- class ServiceEngineerRegistrationViewModel : lưu trữ thông tin ServiceEngineer

```
namespace ASC.Web.Areas.Accounts.Models
{
    2 references
    public class ServiceEngineerRegistrationViewModel
    {
        4 references
        public string UserName { get; set; }

        [Required]
        [EmailAddress]
        [Display(Name = "Email")]
        10 references
        public string Email { get; set; }

        [Required]
        [StringLength(100, ErrorMessage = "The {0} must be at least {2} and at max {1} characters long.", MinimumLength = 6)]
        [DataType(DataType.Password)]
        [Display(Name = "Password")]
        5 references
        public string Password { get; set; }

        [DataType(DataType.Password)]
        [Display(Name = "Confirm password")]
        [Compare("Password", ErrorMessage = "The password and confirmation password do not match.")]
        2 references
        public string ConfirmPassword { get; set; }
        3 references
        public bool IsEdit { get; set; }
        4 references
        public bool IsActive { get; set; }
    }
}
```

- class ServiceEngineerViewModel

```
using Microsoft.AspNetCore.Identity;

namespace ASC.Web.Areas.Accounts.Models
{
    5 references | 0 changes | 0 authors, 0 changes
    public class ServiceEngineerViewModel
    {
        4 references | 0 changes | 0 authors, 0 changes
        public List<IdentityUser>? ServiceEngineers { get; set; } //Lưu trữ danh sách nhân viên
        28 references | 0 changes | 0 authors, 0 changes
        public ServiceEngineerRegistrationViewModel Registration { get; set; } // Lưu trữ nhân viên thêm mới hoặc cập nhật
    }
}
```

B3. Tạo controller AccountController.cs

```
namespace ASC.Web.Areas.Accounts.Controllers
{
    [Authorize]
    [Area("Accounts")]
    1 reference
    public class AccountController : Controller
    {
        private readonly UserManager<IdentityUser> _userManager;
        private readonly IEmailSender _emailSender;
        private readonly SignInManager<IdentityUser> _signInManager;
        0 references
        public AccountController(UserManager<IdentityUser> userManager, IEmailSender emailSender, SignInManager<IdentityUser> signInManager)
        {
            _userManager = userManager;
            _emailSender = emailSender;
            _signInManager = signInManager;
        }
        0 references
        public IActionResult Index()
        {
            return View();
        }
```

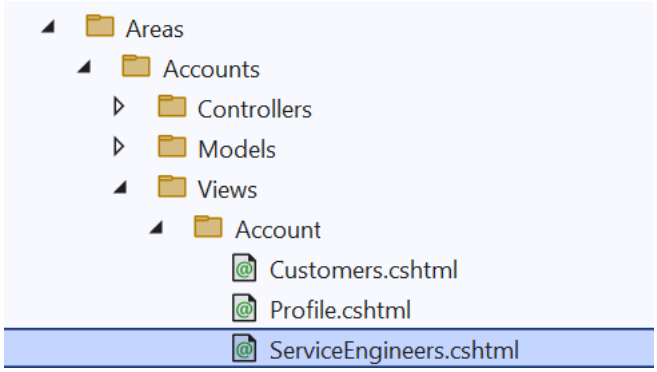
//**Action** ServiceEngineers (httpGet) hiện thị danh sách nhân viên

```
[Authorize(Roles = "Admin")]
[HttpGet]
0 references
public async Task<IActionResult> ServiceEngineers()
{
    var serviceEngineers = await _userManager.GetUsersInRoleAsync(Roles.Engineer.ToString());
    // Hold all service engineers in session
    HttpContext.Session.SetSession("ServiceEngineers", serviceEngineers);
    return View(new ServiceEngineerViewModel
    {
        ServiceEngineers = serviceEngineers == null ? null : serviceEngineers.ToList(),
        Registration = new ServiceEngineerRegistrationViewModel() { IsEdit = false }
    });
}
```

//Action `ServiceEngineers (httpPost) xử lý cập nhật và thêm mới nhân viên`

```
[Authorize(Roles = "Admin")]
[HttpPost]
[ValidateAntiForgeryToken]
0 references
public async Task<IActionResult> ServiceEngineers(ServiceEngineerViewModel serviceEngineer)
{
    serviceEngineer.ServiceEngineers = HttpContext.Session.GetSession<List<IdentityUser>>("ServiceEngineers");
    if (!ModelState.IsValid)
    {
        return View(serviceEngineer);
    }
    if (serviceEngineer.Registration.IsEdit)
    {
        // Update User
        var user = await _userManager.FindByEmailAsync(serviceEngineer.Registration.Email);
        user.UserName = serviceEngineer.Registration.UserName;
        IdentityResult result = await _userManager.UpdateAsync(user);
        if (!result.Succeeded)
        {
            result.Errors.ToList().ForEach(p => ModelState.AddModelError("", p.Description));
            return View(serviceEngineer);
        }
        // Update Password
        var token = await _userManager.GeneratePasswordResetTokenAsync(user);
        IdentityResult passwordResult = await _userManager.ResetPasswordAsync(user, token,serviceEngineer.Registration.Password);
        if (!passwordResult.Succeeded)
        {
            passwordResult.Errors.ToList().ForEach(p => ModelState.AddModelError("",p.Description));
            return View(serviceEngineer);
        }
        // Update claims
        user = await _userManager.FindByEmailAsync(serviceEngineer.Registration.Email);
        var identity = await _userManager.GetClaimsAsync(user);
        var isActiveClaim = identity.SingleOrDefault(p => p.Type == "IsActive");
        var removeClaimResult = await _userManager.RemoveClaimAsync(user,new System.Security.Claims.Claim(isActiveClaim.Type, isActiveClaim.Value));
        var addClaimResult = await _userManager.AddClaimAsync(user,new System.Security.Claims.Claim(isActiveClaim.Type, serviceEngineer.Registration.IsActive.ToString()));
    }
    else
    {
        // Create User
        IdentityUser user = new IdentityUser
        {
            UserName = serviceEngineer.Registration.UserName,
            Email = serviceEngineer.Registration.Email,
            EmailConfirmed = true
        };
        IdentityResult result = await _userManager.CreateAsync(user, serviceEngineer. Registration.Password);
        await _userManager.AddClaimAsync(user, new System.Security.Claims.Claim("http://schemas.xmlsoap.org/ws/2005/05/identity/claims/emailaddress", serviceEngineer.Registration.Email));
        await _userManager.AddClaimAsync(user, new System.Security.Claims.Claim("IsActive",serviceEngineer.Registration.IsActive.ToString()));
        if (!result.Succeeded)
        {
            result.Errors.ToList().ForEach(p => ModelState.AddModelError("",
           p.Description));
            return View(serviceEngineer);
        }
        // Assign user to Engineer Role
        var roleResult = await _userManager.AddToRoleAsync(user, Roles.Engineer.ToString());
        if (!roleResult.Succeeded)
        {
            roleResult.Errors.ToList().ForEach(p => ModelState.AddModelError("",
           p.Description));
            return View(serviceEngineer);
        }
    }
    if (serviceEngineer.Registration.IsActive)
    {
        await _emailSender.SendEmailAsync(serviceEngineer.Registration.Email,"Account Created/Modified",
        $"Email : {serviceEngineer.Registration.Email} /n Passowrd : {serviceEngineer.Registration.Password}");
    }
    else
    {
     await _emailSender.SendEmailAsync(serviceEngineer.Registration.Email,"Account Deactivated",$"Your account has been deactivated.");
    }
    return RedirectToAction("ServiceEngineers");
}
}
}
```

B4. Tạo view cho action `ServiceEngineers.cshtml`



- **Cập nhật _MasterLayout sử dụng datatables**

```
<link rel="stylesheet" type="text/css"
href="https://cdn.datatables.net/1.11.3/css/jquery.dataTables.css">

<script type="text/javascript" charset="utf8"
src="https://cdn.datatables.net/1.11.3/js/jquery.dataTables.js"></script>
```

- **Cập nhật wwwroot/css/ style.css**

```
.padding-bottom-15px {
    padding-bottom: 15px !important;
}

.cursor-hand {
    cursor: pointer
}

.padding-left-20px {
    padding-left: 20px;
}

.padding-right-20px{
    padding-right:20px;
}

.margin-top-50px {
    margin-top: 50px;
}

.margin-bottom-50px {
    margin-bottom: 50px;
}
```

- **Cập nhật view ServiceEngineers.cshtml**
  //Giao diện hiển thị danh sách nhân viên

```
@using Microsoft.AspNetCore.Identity;
@using System.Security.Claims;
@model ASC.Web.Areas.Accounts.Models.ServiceEngineerViewModel
@inject UserManager<IdentityUser> userManager;
@{
    Layout = "_SecureLayout";
}
<div >
    <div class="row"></div>
    <div class="row padding-left-20px padding-right-20px">
        <div class="row z-depth-3">
            <div class="section white-text padding-left-10px blue-grey lighten-1">
                <h5>Users</h5>
            </div>
        </div>
        <div class="divider"></div>
        <div class="col s12 padding-bottom-15px">
            @if (Model.ServiceEngineers != null)
            {
                @* Display List of Service Engineers *@
                <table class="highlight centered" id="tblServiceEngineers">
                    <thead>
                        <tr>
                            <th data-field="Email">Email</th>
                            <th data-field="UserName">Name</th>
                            <th data-field="IsActive">Is Active</th>
                            <th data-field="IsActiveImg">Is Active</th>
                            <th data-field="Actions">Edit User</th>
                        </tr>
                    </thead>
                    <tbody>
                        @foreach (var user in Model.ServiceEngineers)
                        {
                            var identity = await userManager.GetClaimsAsync(user);
                            <tr>
                                <td>@user.Email</td>
                                <td>@user.UserName</td>
                                <td>
                                    @(Boolean.Parse(identity.SingleOrDefault(p => p.Type == "IsActive").Value))
                                </td>
                                <td>
                                    <img src="@(Boolean.Parse(identity.SingleOrDefault(p => p.Type == "IsActive").Value) ? "/images/green_tick.png" : "/images/red_cross.png")" />
                                </td>
                                <td>
                                    <i class="small material-icons edit cursor-hand">
                                        mode_edit
                                    </i>
                                </td>
                            </tr>
                        }
                    </tbody>
                </table>
            }
            else
            {
                @* In case of No records, display no records message *@
                <div class="card blue-grey lighten-1">
                    <div class="card-content white-text">
                        <span class="card-title">No Service Engineers!!!</span>
                        <p>
                            No Service Engineers found, please add a Service Engineer to system.
                        </p>
                    </div>
                </div>
            }
        </div>
        <div class="row"></div>
```

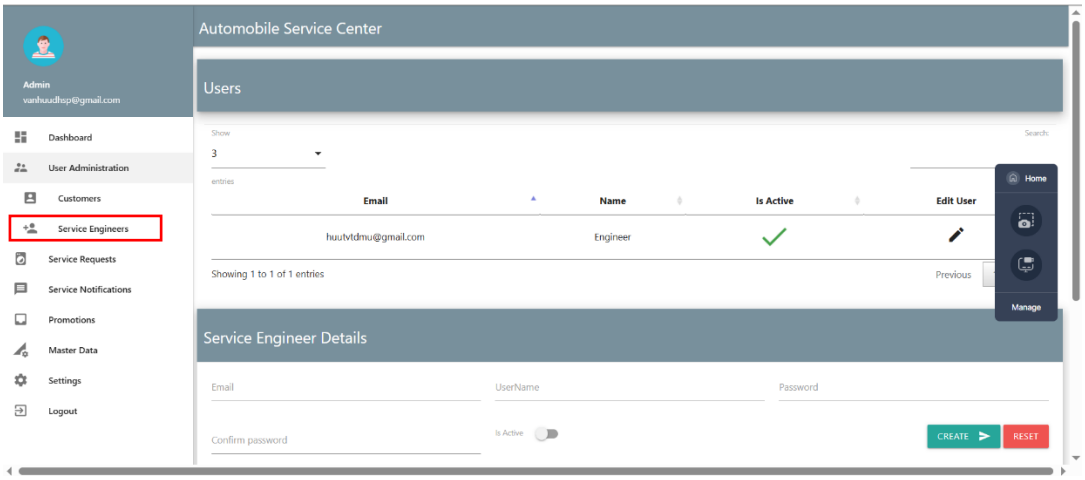
//Giao diện cập nhật và thêm mới nhân viên

```razor
        @* Details Section *@
        <div class="row z-depth-3">
            <div class="col s12 padding-0px">
                <div class="section white-text padding-left-10px blue-grey lighten-1">
                    <h5>Service Engineer Details</h5>
                </div>
                <div class="divider"></div>
                <form asp-controller="Account" asp-action="ServiceEngineers" method="post" class="col s12" id="fromUser">
                    <input type="hidden" asp-for="Registration.IsEdit" />
                    <input type="hidden" asp-for="Registration.Email" />
                    <div class="input-field col s4">
                        <input asp-for="Registration.Email" id="Input_Registration_Email" />
                        <label asp-for="Registration.Email"></label>
                    </div>
                    <div class="input-field col s4">
                        <input asp-for="Registration.UserName" class="validate" />
                        <label asp-for="Registration.UserName"></label>
                    </div>
                    <div class="input-field col s4">
                        <input asp-for="Registration.Password" class="validate" />
                        <label asp-for="Registration.Password"></label>
                    </div>
                    <div class="input-field col s4">
                        <input asp-for="Registration.ConfirmPassword" class="validate" />
                        <label asp-for="Registration.ConfirmPassword"></label>
                    </div>
                    <div class="input-field col s4">
                        <div class="switch">
                            <label>
                                Is Active
                                <input asp-for="Registration.IsActive" class="validate" />
                                <span class="lever"></span>
                            </label>
                        </div>
                    </div>
                    <div class="input-field col s4 right-align">
                        <button class="btn waves-effect waves-light btnSubmit" type="submit" name="action">
                            Create
                            <i class="material-icons right">send</i>
                        </button>
                        <button class="btn waves-effect waves-light reset red lighten-1" type="button" name="action">
                            Reset
                        </button>
                    </div>
                    <div class="row col s12 right-align" asp-validation-summary="All"></div>
                </form>
            </div>
            <div class="row"></div>
        </div>
    </div>
</div>
```

//Script xử lý: datatable, cập nhật , thêm mới nhân viên và reset form

```razor
@section Scripts{
    <script>
        $(document).ready(function () {
            // Initialize DataTable to show list of Engineers
            var table = $('#tblServiceEngineers').DataTable({
                'pageLength': 3,
                // Number of records to be displayed per page
                'lengthMenu': [[3, 5, 10, -1], [3, 5, 10, 'All']]
            });
            // Set Styles for DataTable and Number of Records to be displayed in drop-down
            $('#tblServiceEngineers').css("width", "100%");
            $('select[name="tblServiceEngineers_length"]').formSelect();
            // Get the column API object
            var isActiveColumn = table.column(2);
            isActiveColumn.visible(false);

            // Handle Reset fucntionality
            $(document).on("click", ".reset", function () {
                $('#fromUser')[0].reset();
                $('#Input_Registration_Email').removeAttr('disabled');
                $('.btnSubmit').text('Create');
                $('#Registration_IsEdit').val('False');
            });
            // This is required to disable Email filed on Server-side model validation failure.
            if ($('#Registration_IsEdit').val() === 'True') {
                $('#Input_Registration_Email').attr('disabled', 'disabled');
            }
            // On click of Edit icon, populate the details section with details of service engineer
            $(document).on('click', '.edit', function () {
                var user = $('#tblServiceEngineers').DataTable().row($(this).parents('tr')).data();
                $('#Registration_IsEdit').val('True');
                $('#Input_Registration_Email').val(user[0]);
                $('#Input_Registration_Email').attr('disabled', 'disabled');
                $('#Registration_UserName').val(user[1]);
                $('#Registration_UserName').addClass('valid');
                $('#Registration_IsActive').prop('checked', user[2] === 'True' ? true : false);
                $('.btnSubmit').text('Save');
                M.updateTextFields();
            });
            // Set hidden Email field to correct value from UI field
            // This is required for
            $(document).on('click', '.btnSubmit', function () {
                $('#Registration_Email').val($('#Input_Registration_Email').val());
                M.updateTextFields();
            });
        });
    </script>
}
```

- **Chạy ứng dụng với quyền admin -> User Administrator -> Service Engineers**



III. Use Case Customer Dashboard – Role Admin : quản lý tài khoản người dùng. Hiện thị danh sách người dùng. Và cho phép cập nhật trạng thái tài khoản người dùng active hoặc deactive. Khi cập nhật thông tin tài khoản phải gửi thống báo đến người dùng qua email.

B1. Tạo model

- class CustomerRegistrationViewModel : lưu trữ thông tin khách hàng đăng ký

```
namespace ASC.Web.Areas.Accounts.Models
{
    1 reference
    public class CustomerRegistrationViewModel
    {
        [Required]
        [EmailAddress]
        [Display(Name = "Email")]
        3 references
        public string Email { get; set; }

        0 references
        public string UserName { get; set; }
        1 reference
        public bool IsEdit { get; set; }
        1 reference
        public bool IsActive { get; set; }
    }
}
```

- class CustomerViewModel : lưu trữ danh sách khách hàng và khách hàng đăng ký

```
using Microsoft.AspNetCore.Identity;

namespace ASC.Web.Areas.Accounts.Models
{
    5 references | 0 changes | 0 authors, 0 changes
    public class CustomerViewModel
    {
        4 references | 0 changes | 0 authors, 0 changes
        public List<IdentityUser>? Customers { get; set; }
        13 references | 0 changes | 0 authors, 0 changes
        public CustomerRegistrationViewModel Registration { get; set; }
    }
}
```

B2. Cập nhật Account controller

- **Thêm mới action** `Customers (httpGet) để lấy danh sách khách hàng`

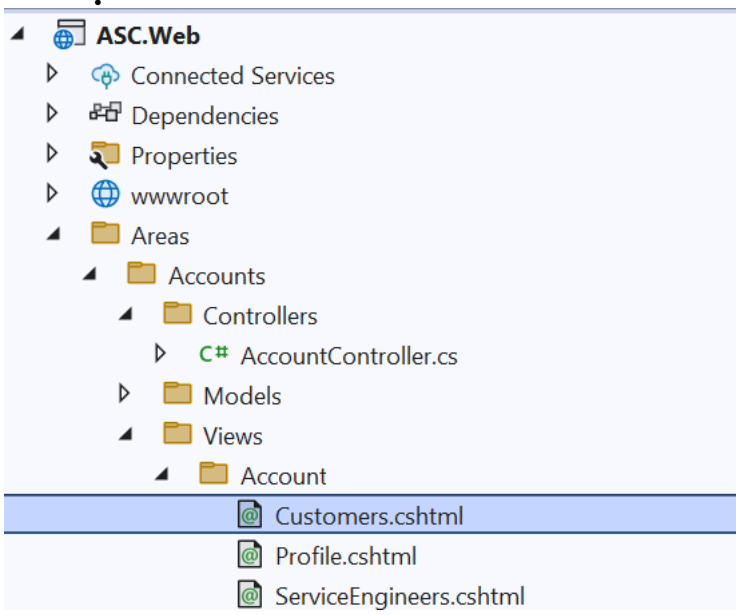
```
[HttpGet]
0 references
public async Task<IActionResult> Customers()
{
    var customers = await _userManager.GetUsersInRoleAsync(Roles.User.ToString());
    // Hold all service engineers in session
    HttpContext.Session.SetSession("Customers", customers);
    return View(new CustomerViewModel
    {
        Customers = customers == null ? null : customers.ToList(),
        Registration = new CustomerRegistrationViewModel() { IsEdit = false }
    });
}
```

- **Thêm mới action** `Customers (httpPost) để cập nhật trạng thái khách hàng sang active và deactive`

  Chú ý: không có action xử lý thêm mới khách hàng. Mà khách hàng phải đăng ký qua giao diện người dùng

```
        [HttpPost]
        [ValidateAntiForgeryToken]
        0 references
        public async Task<IActionResult> Customers(CustomerViewModel customer)
        {
            customer.Customers = HttpContext.Session.GetSession<List<IdentityUser>>("Customers");
            if (!ModelState.IsValid)
            {
                return View(customer);
            }

            if (customer.Registration.IsEdit)
            {
                // Update User
                // Update claims IsActive
                var user = await _userManager.FindByEmailAsync(customer.Registration.Email);
                var identity = await _userManager.GetClaimsAsync(user);
                var isActiveClaim = identity.SingleOrDefault(p => p.Type == "IsActive");
                var removeClaimResult = await _userManager.RemoveClaimAsync(user, new System.Security.Claims.Claim(isActiveClaim.Type, isActiveClaim.Value));
                var addClaimResult = await _userManager.AddClaimAsync(user, new System.Security.Claims.Claim(isActiveClaim.Type, customer.Registration.IsActive.ToString())
            }

            if (customer.Registration.IsActive)
            {
                await _emailSender.SendEmailAsync(customer.Registration.Email, "Account Modified",$"Your account has been activated, Email : {customer.Registration.Email}"
            }
            else
            {
                await _emailSender.SendEmailAsync(customer.Registration.Email, "Account Deactivated", $"Your account has been deactivated.");
            }
            return RedirectToAction("Customers");
        }


    }
}
```

B3. Tạo view cho action Customers.cshtml



//Hiển thị danh sách khách hàng

```
@using Microsoft.AspNetCore.Identity;
@using System.Security.Claims;
@model ASC.Web.Areas.Accounts.Models.CustomerViewModel
@inject UserManager<IdentityUser> userManager;
@{
    Layout = "_SecureLayout";
}
<div >
    <div class="row"></div>
    <div class="row padding-left-20px padding-right-20px">
        <div class="row z-depth-3">
            <div class="section white-text padding-left-10px blue-grey lighten-1">
                <h5>Users</h5>
            </div>
        </div>
        <div class="divider"></div>
        <div class="col s12 padding-bottom-15px">
            @if (Model.Customers != null)
            {
                @* Display List of Service Engineers *@
                <table class="highlight centered" id="tblCustomers">
                    <thead>
                        <tr>
                            <th data-field="Email">Email</th>
                            <th data-field="IsActive">Is Active</th>
                            <th data-field="IsActiveImg">Is Active</th>
                            <th data-field="Actions">Edit User</th>
                        </tr>
                    </thead>
                    <tbody>
                        @foreach (var user in Model.Customers)
                        {
                            var identity = await userManager.GetClaimsAsync(user);
                            <tr>
                                <td>@user.Email</td>
                                <td>
                                    @(Boolean.Parse(identity.SingleOrDefault(p => p.Type == "IsActive").Value))
                                </td>
                                <td>
                                    <img src="@(Boolean.Parse(identity.SingleOrDefault(p => p.Type == "IsActive").Value) ? "/images/green_tick.png" : "/images/red_cross.png")" />
                                </td>
                                <td>
                                    <i class="small material-icons edit cursor-hand">
                                        mode_edit
                                    </i>
                                </td>
                            </tr>
                        }
                    </tbody>
                </table>
            }
            else
            {
                @* In case of No records, display no records message *@
                <div class="card blue-grey lighten-1">
                    <div class="card-content white-text">
                        <span class="card-title">No Service Engineers!!!</span>
                        <p>
                            No Service Engineers found, please add a Service Engineer to system.
                        </p>
                    </div>
                </div>
            }
        </div>
        <div class="row"></div>
```

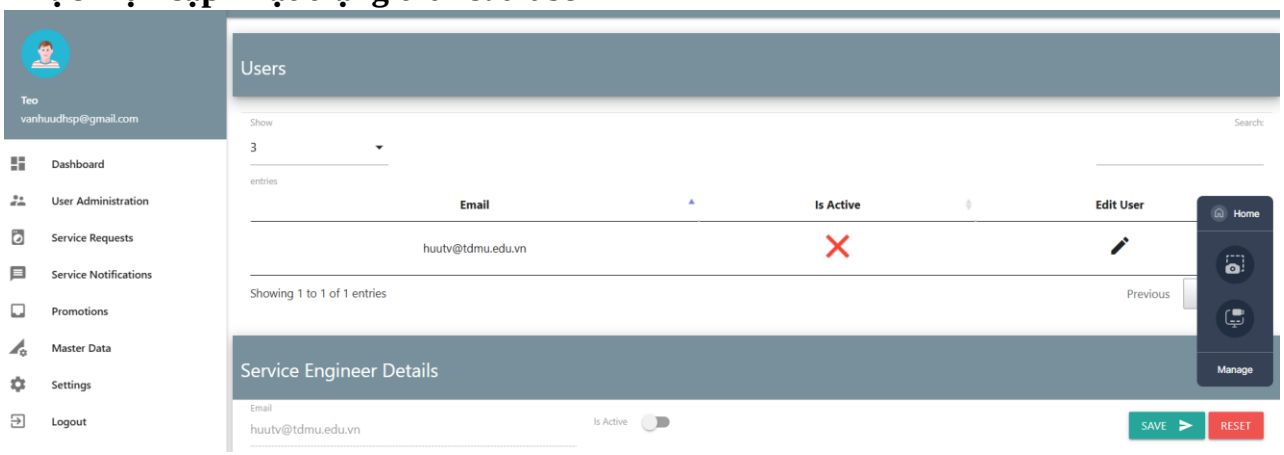
//View cập nhật khách hàng

```
        @* Details Section *@
        <div class="row z-depth-3">
            <div class="col s12 padding-0px">
                <div class="section white-text padding-left-10px blue-grey lighten-1">
                    <h5>Service Engineer Details</h5>
                </div>
                <div class="divider"></div>
                <form asp-controller="Account" asp-action="Customers" method="post" class="col s12" id="fromUser">
                    <input type="hidden" asp-for="Registration.IsEdit" />
                    <input type="hidden" asp-for="Registration.Email" />
                    <div class="input-field col s4">
                        <input asp-for="Registration.Email" id="Input_Registration_Email" />
                        <label asp-for="Registration.Email"></label>
                    </div>
                    <div class="input-field col s4">
                        <div class="switch">
                            <label>
                                Is Active
                                <input asp-for="Registration.IsActive" class="validate" />
                                <span class="lever"></span>
                            </label>
                        </div>
                    </div>
                    <div class="input-field col s4 right-align">
                        <button class="btn waves-effect waves-light btnSubmit" type="submit" name="action">
                            Save
                            <i class="material-icons right">send</i>
                        </button>
                        <button class="btn waves-effect waves-light reset red lighten-1" type="button" name="action">
                            Reset
                        </button>
                    </div>
                    <div class="row col s12 right-align" asp-validation-summary="All"></div>
                </form>
            </div>
            <div class="row"></div>
        </div>
    </div>
</div>
```

//Script xử lý các sự kiện

```
@section Scripts{
    <script>
        $(document).ready(function () {
            // Initialize DataTable to show list of Engineers
            var table = $('#tblCustomers').DataTable({
                'pageLength': 3,
                // Number of records to be displayed per page
                'lengthMenu': [[3, 5, 10, -1], [3, 5, 10, 'All']]
            });
            // Set Styles for DataTable and Number of Records to be displayed in drop-down
            $('#tblCustomers').css("width", "100%");
            $('select[name="tblCustomers_length"]').formSelect();
            // Get the column API object
            var isActiveColumn = table.column(1);
            isActiveColumn.visible(false);
            $('.btnSubmit').attr('disabled', 'disabled');

            // Handle Reset fucntionality
            $(document).on("click", ".reset", function () {
                $('#fromUser')[0].reset();
                $('#Input_Registration_Email').removeAttr('disabled');
                $('#Registration_IsEdit').val('False');
                $('.btnSubmit').attr('disabled', 'disabled');
            });
            // This is required to disable Email filed on Server-side model validation failure.
            if ($('#Registration_IsEdit').val() === 'True') {
                $('#Input_Registration_Email').attr('disabled', 'disabled');
                $('.btnSubmit').removeAttr('disabled');
            }
            // On click of Edit icon, populate the details section with details of service engineer
            $(document).on('click', '.edit', function () {
                var user = $('#tblCustomers').DataTable().row($(this).parents('tr')).data();
                $('#Registration_IsEdit').val('True');
                $('#Input_Registration_Email').val(user[0]);
                $('#Input_Registration_Email').attr('disabled', 'disabled');
                $('#Registration_IsActive').prop('checked', user[1] === 'True' ? true : false);
                $('.btnSubmit').removeAttr('disabled');
                M.updateTextFields();
            });
            // Set hidden Email field to correct value from UI field
            // This is required for
            $(document).on('click', '.btnSubmit', function () {
                $('#Registration_Email').val($('#Input_Registration_Email').val());
                M.updateTextFields();
            });
        });
    </script>
}
```

B4. Chạy ứng dụng role admin -> User Aministration -> Customers
Thực hiện cập nhật trạng thái của user



IV. User Case Update Profile: thực hiện cập nhật username cho người dùng

B1. Tạo các model
/Areas/Accounts/Models/ProfileModel.cs

```
namespace ASC.Web.Areas.Accounts.Models
{
    5 references | 0 changes | 0 authors, 0 changes
    public class ProfileModel
    {
        4 references | 0 changes | 0 authors, 0 changes
        public string UserName { get; set; }
    }
}
```

B2. Thêm mới Action Get Profile trong /Areas/Accounts/Controllers/ AccountController.cs

```
[HttpGet]
0 references | 0 changes | 0 authors, 0 changes
public IActionResult Profile()
{
    var user = HttpContext.User.GetCurrentUserDetails();

    return View(new ProfileModel() { UserName = user.Name });
}
```

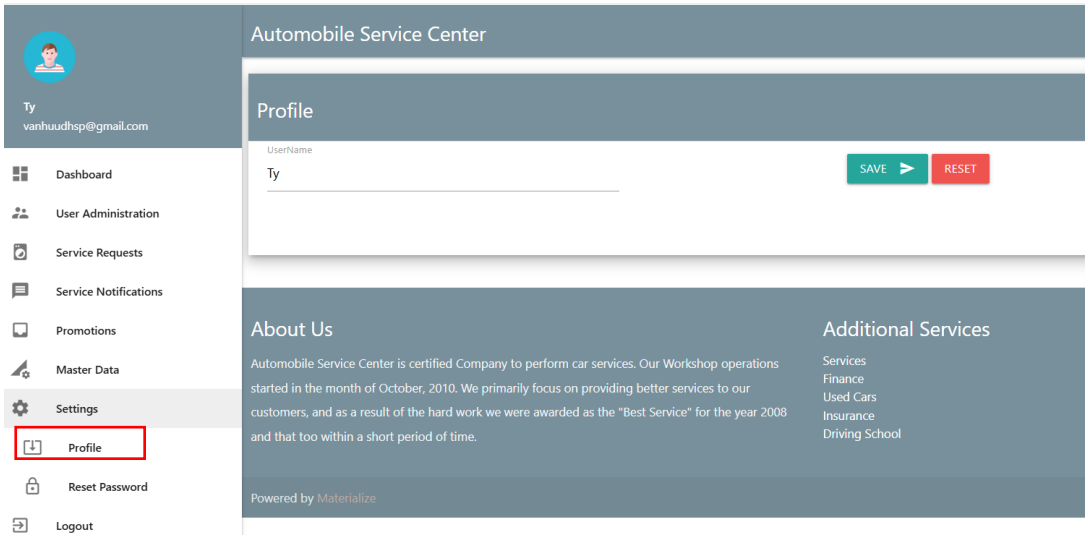
B3. Tạo view cho action Profile /Areas/Accounts/Views/ Account/Profile.cshtml

```
@model ASC.Web.Areas.Accounts.Models.ProfileModel
@{
    Layout = "_SecureLayout";
}
<div>
    <div class="row"></div>
    <div class="row padding-left-20px padding-right-20px">
        @* Details Section *@
        <div class="row z-depth-3">
            <div class="col s12 padding-0px">
                <div class="section white-text padding-left-10px blue-grey lighten-1">
                    <h5>Profile</h5>
                </div>
                <div class="divider"></div>
                <form asp-controller="Account" asp-action="Profile" method="post" class="col s12" id="fromUser">
                    <div class="input-field col s4">
                        <input asp-for="UserName" class="validate" />
                        <label asp-for="UserName"></label>
                    </div>

                    <div class="input-field col s4 right-align">
                        <button class="btn waves-effect waves-light btnSubmit" type="submit" name="action">
                            Save
                            <i class="material-icons right">send</i>
                        </button>
                        <button class="btn waves-effect waves-light reset red lighten-1" type="button" name="action">
                            Reset
                        </button>
                    </div>
                    <div class="row col s12 right-align" asp-validation-summary="All"></div>
                </form>
            </div>
            <div class="row"></div>
        </div>
    </div>
</div>

@section Scripts {
    <script>
        $(document).ready(function () {
            // Handle Reset fucntionality
            $(document).on("click", ".reset", function () {
                $('#fromUser')[0].reset();
                /*          $('.btnSubmit').text('Create');*/
            });

            // Set hidden Email field to correct value from UI field
            // This is required for
            $(document).on('click', '.btnSubmit', function () {
                M.updateTextFields();
            });
        });
    </script>
}
```

B4. Thêm mới Action Post Profile trong /Areas/Accounts/Controllers/ AccountController.cs

```
[HttpPost]
[ValidateAntiForgeryToken]
0 references | 0 changes | 0 authors, 0 changes
public async Task<IActionResult> Profile(ProfileModel profile)
{
    if (!ModelState.IsValid)
    {
        return View();
    }

    // Update UserName
    var user = await _userManager.FindByEmailAsync(HttpContext.User.GetCurrentUserDetails().Email);
    user.UserName = profile.UserName;
    IdentityResult result = await _userManager.UpdateAsync(user);
    if (!result.Succeeded)
    {
        result.Errors.ToList().ForEach(p => ModelState.AddModelError("", p.Description));
        return View();
    }
    await _signInManager.RefreshSignInAsync(user);

    return RedirectToAction("Dashboard", "Dashboard", new {area = "ServiceRequests" });
}
```

B5. Thực hiện chạy và cập nhật Profile


Automobile Service Center
Ty
vanhuudhsp@gmail.com
Dashboard
User Administration
Service Requests
Service Notifications
Promotions
Master Data
Settings
Profile
Reset Password
Logout
Profile
UserName
Ty
SAVE
RESET
About Us
Automobile Service Center is certified Company to perform car services. Our Workshop operations started in the month of October, 2010. We primarily focus on providing better services to our customers, and as a result of the hard work we were awarded as the "Best Service" for the year 2008 and that too within a short period of time.
Additional Services
Services
Finance
Used Cars
Insurance
Driving School
Powered by Materialize